LAB 08: GRAPH BASIC

CSE 4304

Data Structures Lab

Mahmudul Islam Mahin
210042140

Department of CSE
B.Sc in Software Engineering

November 5, 2023

Contents

Introduction

In Data Structures, a "Graph" is a fundamental and versatile abstract data type used to represent and analyze relationships and connections between objects. It is composed of two main components: nodes (also called vertices) and edges.

BFS: Breadth-First Search

- BFS is an algorithm for traversing or searching a graph, and it is often used to find the shortest path in an unweighted graph.
- It explores all the neighbor nodes at the current depth level before moving on to nodes at the next level.
- BFS is typically implemented using a queue data structure to ensure that nodes are processed in a first-in-first-out (FIFO) manner.

DFS: Depth-First Search

- DFS is another algorithm for traversing or searching a graph, but it explores as far as possible along each branch before backtracking.
- It is often implemented using a stack (or recursion in the case of a recursive implementation) to handle the backtracking.

Dijkstra's Algorithm

- Dijkstra's algorithm is used to find the shortest path in a weighted graph with non-negative edge weights.
- It maintains a set of nodes with tentative distances and repeatedly selects the node with the smallest tentative distance and relaxes the distances of its neighbors.
- Dijkstra's algorithm ensures that it finds the shortest path from a source node to all other nodes in the graph.
- It's commonly implemented using a priority queue or a min-heap data structure to efficiently select the node with the smallest tentative distance.

1 Back to Underworld

In this scenario, there are two distinct types of fighters: vampires and lycans. These fighters engage in duels, and the winner of each duel proceeds to face a fighter of the opposite type.

However, initially, we have no information about which fighter is a vampire and which one is a lycan. To address this uncertainty, we employ a strategy involving BFS (Breadth-First Search). We randomly designate one fighter as a vampire and all the fighters they've battled as lycans, or we can do the reverse, designating one fighter as a lycan and the rest as vampires.

The key aspect of this strategy is that, after each BFS operation, we evaluate the results. In other words, we update the outcome based on the assignments we make during the BFS traversal. This process helps us determine the best way to maximize the number of fighters of the same type in the given dueling context.

1.1 Code Snippet

Listing 1: A

```
#include<bits/stdc++.h>
typedef long long int ll;
using namespace std;
int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int t;
    cin>>t;
    for(int i=1; i<=t; i++)
    {
        vector<int>adj[20008];
        int vis[20008]= {0};
        int type[20008]= {0};
        set<int>s;

        int n;
        cin>>n;
        while (n--)
        {
            int a,b;
            cin>>a>>b;
            adj[a].push_back(b);
            adj[b].push_back(a);
            s.insert(a);
```

```
            s.insert(b);
        }
        ll ans=0;

        for(auto i:s)
        {
            if(!vis[i])
            {
                queue<int>q;
                q.push(i);
                vis[i]=1;
                type[i]=1;
                int cnt1=1,cnt2=0;
                while(!q.empty())
                {
                    int u=q.front();
                    q.pop();
                    for(int v:adj[u])
                    {
                        if(!vis[v])
                        {
                            vis[v]=1;
                            q.push(v);
                            if(type[u]==1)
                            {
                                type[v]=2;
                                cnt2++;
                            }
                            else
                            {
                                type[v]=1;
                                cnt1++;
                            }
                        }
                    }
                }
                ans+=max(cnt1,cnt2);
            }
        }
        cout<<"Case "<<i<<": ";
        cout<<ans<<"\n";
    }
}
```

2 Kefa and Park

To explore the entire graph, a Depth-First Search (DFS) approach is employed. During the traversal, a count of consecutive nodes with a certain property, such as cats, is maintained. If this count surpasses a predefined threshold (mxcat), the traversal path is terminated.

While progressing through the graph, when reaching a leaf node (a node with no further connections), the answer is incremented. This increase in the answer represents the identification of restaurant locations, which are situated at the leaf nodes.

2.1 Code Snippet

Listing 2: B

```
#include<bits/stdc++.h>
using namespace std;
const int maxn = 1e5+100;
vector<int> node[maxn];
int cats[maxn],vis[maxn];
int n,mxcat,u,v,ans=0;
void dfs(int x,int st)
{
    if(vis[x]==1) return;
    vis[x]=1;
    if(cats[x]) st++;
    else st=0;
    if(st>mxcat) return;
    if(node[x].size()==1 && x!=1)
    {
        ans++;
        return ;
    }
    for(int i = 0;i<node[x].size();i++)
    {
        dfs(node[x][i],st);
    }
}
int main()
{
    cin>>n>>mxcat;
    for(int i = 1;i<=n;i++)
    {
        cin>>cats[i];
    }
    for(int i = 1;i<n;i++)
```

```
    {
        cin>>u>>v;
        node[u].push_back(v);
        node[v].push_back(u);
    }
    dfs(1,0);
    cout<<ans;
    return 0;
}
```

3 Oil Deposits

The problem at hand is a straightforward DFS (Depth-First Search) challenge. We iterate through all the cells using nested loops. Whenever we come across a cell containing "@" that hasn't been visited previously, we initiate a DFS call for that cell.

The final answer is determined by counting how many times the DFS function is invoked, representing the total number of connected regions containing "@" symbols.

3.1 Code Snippet

Listing 3: C

```
#include <iostream>
#include <vector>
using namespace std;
int m, n;
vector<vector<char>> grid;
vector<vector<bool>> visited;
const int dx[] = {1, 1, 1, 0, 0, −1, −1, −1};
const int dy[] = {1, 0, −1, 1, −1, 1, 0, −1};
bool isValid(int x, int y)
{
    return x >= 0 && x < m && y >= 0 && y < n;
}

void DFS(int x, int y)
{
    visited[x][y] = true;
    for (int i = 0; i < 8; i++)
    {
        int newX = x + dx[i];
        int newY = y + dy[i];

        if (isValid(newX, newY) && !visited[newX][newY] && grid[newX][newY] ==
            '@')
        {
            DFS(newX, newY);
        }
    }
}

int countOilDeposits()
{
    visited.assign(m, vector<bool>(n, false));
```

```
    int oilDeposits = 0;

    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (!visited[i][j] && grid[i][j] == '@')
            {
                oilDeposits++;
                DFS(i, j);
            }
        }
    }

    return oilDeposits;
}

int main()
{
    int gridNum = 1;
    while (true)
    {
        cin >> m >> n;
        if (m == 0)
        {
            break;
        }

        grid.assign(m, vector<char>(n));

        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                cin >> grid[i][j];
            }
        }

        int oilDeposits = countOilDeposits();
        cout << "Grid " << gridNum << ": " << oilDeposits << " oil deposits" <<
            endl;
        gridNum++;
    }
    return 0;
}
```

4 Travelling Cost

In the context of a weighted graph, solving this problem necessitates the use of Dijkstra's algorithm. Initially, all nodes are assigned an infinite distance. The next step involves applying Dijkstra's algorithm to the source node, which leads to an update of the distances for all reachable nodes.
The answer can be expressed as follows: If the distance of a node remains infinite, this indicates that there is no path to reach that node. On the other hand, if the distance of a node is not infinite, the answer for that node is equal to its distance value.

4.1 Code Snippet

Listing 4: D

```
#include<bits/stdc++.h>
typedef long long int ll;
using namespace std;
const int N=505;
const int INF=1e9+10;

vector<pair<int,int>>g[N];
vector<int>dist(N,INF);
vector<int>vis(N,0);

void dijkstra(int source)
{
    set<pair<int,int>>st;

    st.insert({0,source});
    dist[source]=0;

    while(st.size()>0)
    {
        auto node=*st.begin();
        int v=node.second;
        int v_dist=node.first;
        st.erase(st.begin());
        if(vis[v]) continue;

        vis[v]=1;
        for(auto child:g[v])
        {
            int child_v=child.first;
            int wt=child.second;
```

```
            if(dist[v]+wt <dist[child_v])
            {
                dist[child_v] = dist[v]+wt;
                st.insert({dist[child_v],child_v});
            }
        }
    }
}

int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int m;
    cin>>m;
    for(int i=0;i<m;i++)
    {
        int x,y,wt;
        cin>>x>>y>>wt;
        g[x].push_back({y,wt});
        g[y].push_back({x,wt});
    }

    int u;
    cin>>u;
    dijkstra(u);

    int q;
    cin>>q;
    while(q--)
    {
        int v;
        cin>>v;
        if(dist[v]==INF)
        {
            cout<<"NO PATH\n";
        }
        else
        {
            cout<<dist[v]<<"\n";
        }
    }
}
```